
Computer Organization & Architecture(25TU03MJA0)

1. Lab Instructions:

- I. Follow all instructions carefully in the lab.
- II. Without permission mobile phone is not allowed.
- III. Formals, shoes and ID cards are compulsory for the lab.
- IV. Don't install or delete any software without permission.
- V. Students should refer to the laboratory manual and prepare for the planned experiment before coming to the lab every week.
- VI. Students should complete writing their experiment records before arriving at the laboratory and have them reviewed and signed by the Lab Instructor.

2. Lab Exam Instructions and Evaluation:

- I. A total of 10 experiments, as outlined in the provided manual, collectively account for 100 marks.
- II. There will be two separate examinations. The first examination will cover the first 5 experiments, and the second examination will assess the remaining 5 experiments. By default, these examinations will take place in the week following the completion of each set of 5 experiments.
- III. During the examination, a single experiment will be randomly selected from these six for the exam evaluation.
- IV. Each examination carries a total of 50 marks, distributed as follows:

3. Course Objectives: Upon successful completion, Students will have

- To impart basic concepts of this course provides a foundational understanding of computer

organization and architecture concepts, circuit designs, and functioning of computer.

- To Learn and explore the concepts of binary arithmetic's, hard wired control designs, memory and I/O circuits.

Course Learning Outcomes:

CILO 1	To study the design of arithmetic operations and implementation.
CILO 2	Understanding basic processing unit and organization of processor including instruction sets, instruction formats and various addressing modes.
CILO 3	Analyze memory operations in a way to facilitate both speed and time.

Module:

Module No.	Topics	CLO Mapping
Unit 1	Computer Design	CLO 1
Unit 2	Arithmetic	CLO 2
Unit 3	Processor	CLO 3
Unit 4	Control Unit	CLO 3
Unit 5	Memory unit & I/O	CLO 2

Text Books:

1. “Computer Organization and Architecture: Designing for Performance”, 10th Edition by William Stallings, Pearson Education.
2. “Computer System Architecture”, M. Morris Mano, Pearson

Reference Books:

1. “Computer Architecture and Organization”, 3rd Edition by John P. Hayes, WCB/McGraw Hill
2. “Computer System Design and Architecture”, 2nd Edition by Vincent P. Heuring and Harry F. Jordan, Pearson Education

E-content:

1. <https://de-iitr.vlabs.ac.in/List%20of%20experiments.html>

Attendance Policy:

1. A Student must normally maintain a minimum of 75% attendance in the course.
2. Lab without which he/she shall be disqualified from appearing in the respective

lab examination.

Evaluation Scheme:

Lab Exam – I:

Maximum Marks: 50 Mark

Sr.no.	Marks of Distribution	Marks
1	Performing Experiment [Aim (1 mark), Introduction (3 marks), Circuit Diagram (10 marks), Truth table (5 marks), Simulation and Results (output) (10 marks), Conclusion (1 mark)]	30
2	Record Books	10
3	Continuous Attendance	05
4	Viva	05

Lab Exam – II: Maximum Marks: 50 Marks

Sr.no.	Marks of Distribution	Marks
1	Performing Experiment [Aim (1 mark), Introduction (3 marks), Circuit Diagram (10 marks), Truth table (5 marks), Simulation and Results (output) (10 marks), Conclusion (1 mark)]	30
2	Record Books	10
3	Continuous Attendance	05
4	Viva	05

INDEX

S.no	Experiment Names	Page no	CIO Mapping
1	Study of Logisim Tool.	7 – 14	CILO 1
2	Realization of Logic Gates.	15 – 21	CILO 1
3	Construction of half/ full adder using XOR and NAND gates and verification of its operation.	22 – 36	CILO 1
4	Study and Verify Half and Full Subtractor.	37 – 44	CILO 1
5	Implementation and verification of decoder or de-multiplexer and encoder using logic gates.	45 – 52	CILO 2
6	Implementation of 4x1 multiplexer and 1x4 demultiplexer using logic gates.	53 – 59	CILO 2
7	Verify the truth table of RS, JK, T and D flip-flops using NAND and NOR gates.	60 – 69	CILO 3
8	Design and verify the 4-Bit Serial In - Parallel Out Shift Registers.	70 – 76	CILO 3
9	To analyze the circuit and the truth table of 4-bit SIPO shift register by using the JK Flip Flop	77 – 78	CILO 3
10	To study and implement a 7-segment display using a BCD to 7-segment decoder and observe the display of decimal digits (0–9)	79 – 93	CILO 2

Experiment No:1

1. Aim:

To Study the Logisim Tool.

2. Introduction:

Logisim allows you to design and simulate digital circuits. It is intended as an educational tool, to help you learn how circuits work. To practice using Logisim, let's build a XOR circuit - that is, a circuit that takes two inputs (which we'll call x and y) and outputs 0 if the inputs are the same and 1 if they are different. The following truth table illustrates.

Table 1.1: Truth Table of $x \text{ XOR } y$

x	y	$x \text{ XOR } y$
0	0	0
0	1	1
1	0	1
1	1	0

We might design such a circuit on paper.

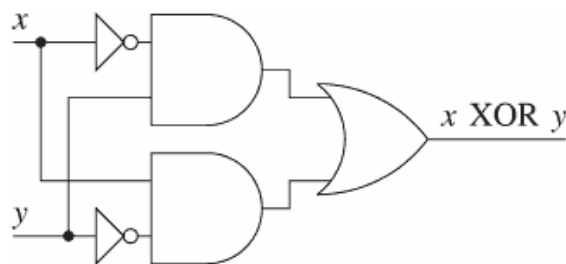


Fig 1.1: XOR Circuit Diagram

But just because it's on paper doesn't mean it's right. To verify our work, we'll draw it in Logisim and test it.

Step 1: Adding gates

Building a circuit is easiest by inserting the gates first as a sort of skeleton for connecting wires into the circuit later. The first thing we're going to do is to add the two AND gates. Click on the AND tool in the toolbar (D, the next-to-last tool listed). Then click in the editing area where you want the AND gates to go. Be sure to leave plenty of room for stuff on the left.

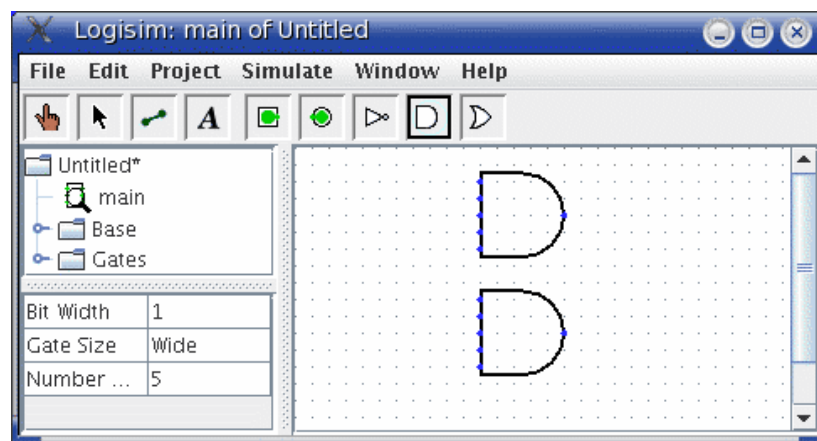


Fig 1.2: Taking 2 AND gates

Notice the five dots on the left side of the AND gate. These are spots where wires can be attached. It happens that we'll just use two of them for our XOR circuit; but for other circuits, you may find that having more than two wires going to an AND gate is useful.

Now add the other gates. First click on the OR tool; then click where you want it. And select the NOT tool and put those two gates into the canvas.

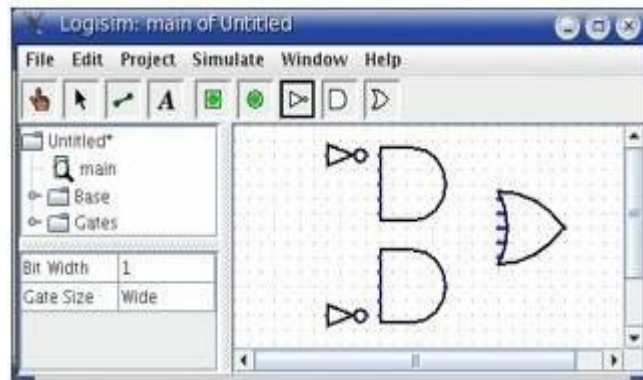


Fig 1.3: Considering 2 NOT and 1 OR gate

I left a little space between the NOT gates and the AND gates; if you want to, though, you can put them up against each other and save yourself the effort of drawing a wire in later.

Now we want to add the two inputs x and y into the diagram. Select the input pin, and place the pins down. You should also place an output pin, next to the OR gate's output. (Again, though I'm leaving a bit of space between the OR gate and the output pin, you might choose to place them right next to each other.)

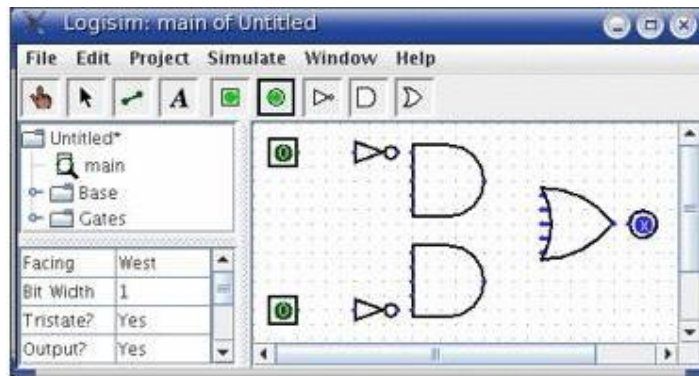


Fig 1.4: Adding 2 Input/Output pins

If you decide you don't like where you placed something, then you can right-click (or control-click) anything in the canvas to view a pop-up menu. Choose Delete. You can also rearrange things using the select tool.

Step 2: Adding Wires

After you have all the components blocked out on the canvas, you're ready to start adding wires. Select the wiring tool. Then start dragging from one position to another in the canvas area, and a wire will start to appear between the two points. The wiring tool is pretty intelligent. Whenever a wire ends at another wire, Logisim automatically connects them. You can also "extend" or "shorten" a wire by dragging one of its endpoints using the wiring tool.

Wires in Logisim must be horizontal or vertical. To connect the upper input to the NOT gate and the AND gate, then, I added three different wires. Logisim automatically connects wires to the gates and to each other. This includes automatically drawing the circle at a T intersection as above, indicating that the wires are connected.

As you draw wires, you may see some blue or gray wires. Blue in Logisim indicates that the value at that point is “unknown”, and gray indicates that the wire is not connected to anything. This is not a big deal temporarily. But by the time you finish your circuit, none of your wires should be blue or gray. (The unconnected legs of the OR gate will still be blue: That's fine.)

If you do have a blue or a gray wire after you think everything ought to be connected, then something is going wrong. It's important that you connect wires to the right places. Logisim draws little dots on the components to indicate where wires ought to connect. As you proceed, you'll see the dots turn from blue to light or dark green.

Once you have all the wires connected, all of the wires you inserted will themselves be light or dark green.

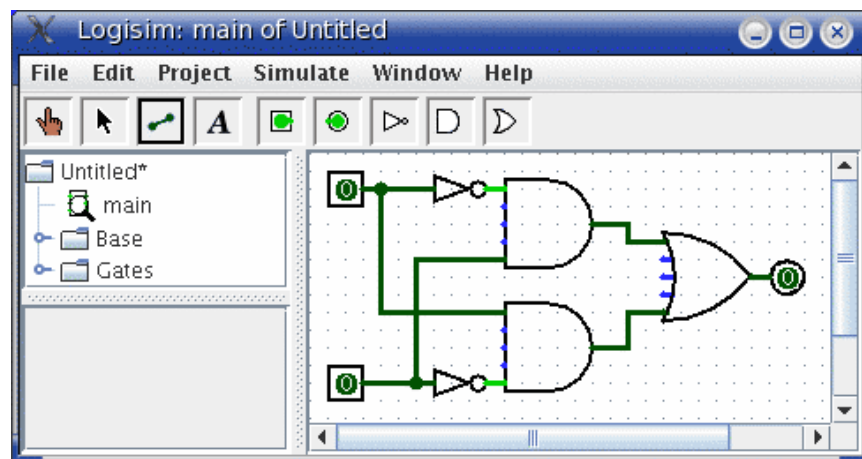


Fig 1.5: Connecting the gates with wires

Step 3: Adding text

Adding text to the circuit isn't necessary to make it work; then some labels help to communicate the purpose of the different pieces of your circuit.

Select the text tool (**A**). You can click on an input pin and start typing to give it a label. (It's better to click directly on the input pin than to click where you want the text to go, because then the label will move with the pin.) You can do the same for the output pin. Or you could just click any old place and start typing to put a label anywhere else.

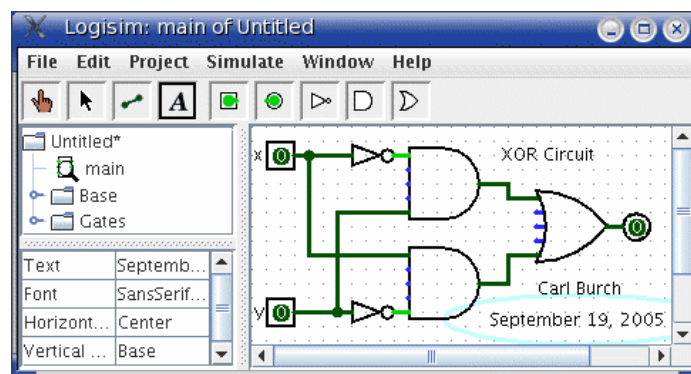


Fig 1.6: Add text to the circuit

Step 4: Testing your circuit

Our final step is to test our circuit to ensure that it really does what we intended. Logisim is already simulating the circuit. Let's look again at where we were.

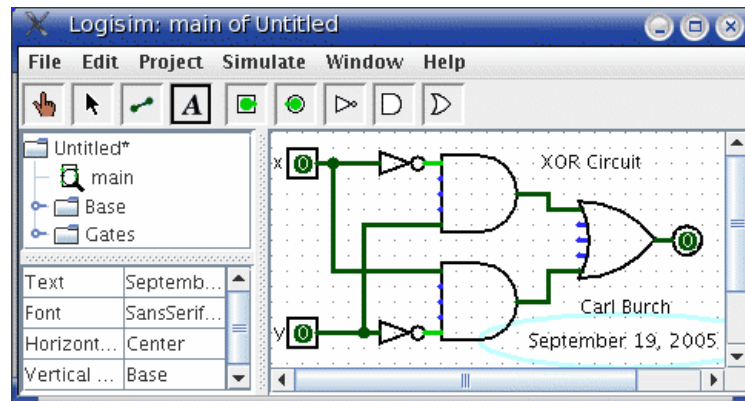


Fig 1.7: Test the circuit with Binary Input 1/0

Note that the input pins both contain 0s; and so, does the output pin. This already tells us that the circuit already computes a 0 when both inputs are 0. Now to try another combination of inputs. Select the poke tool () and start poking the inputs by clicking on them. Each time you poke an input, its value will toggle. For example, we might first poke the bottom input.

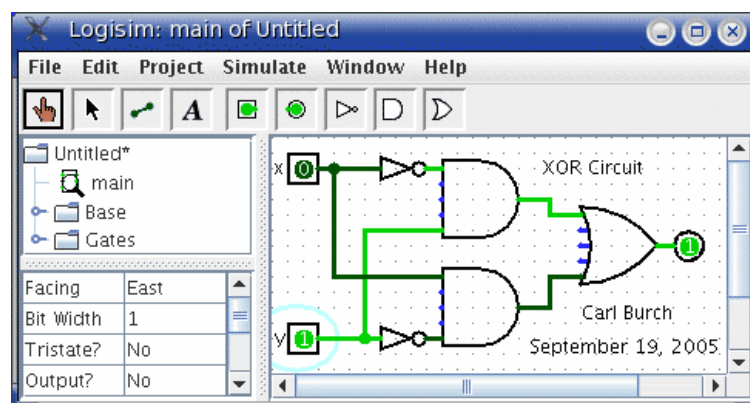


Fig 1.8: A completed x XOR y working circuit

When you change the input value, Logisim will show you what values travel down the wires by drawing them light green to indicate a 1 value or dark green (almost black) to indicate a 0 value. You can also see that the output value has changed to 1.

So far, we have tested the first two rows of our truth table, and the outputs (0 and 1) match the desired outputs.

x	y	$x \text{ XOR } y$
0	0	0
0	1	1
1	0	1
1	1	0

Table 1.2: Truth Table verification of $x \text{ XOR } y$

By poking the switches through different combinations, we can verify the other two rows. If they all match, then we're done: The circuit works.

3. Conclusion:

Hence the Logisim tool has been studied in a detailed way with an example of $x \text{ XOR } y$.

4. Reference:

1. <http://www.cburch.com/logisim/docs/1.0/tutor.html>
2. https://www.youtube.com/watch?v=cMz7wyY_PxE

Experiment No: 2

1. Aim:

To realize the logic gates using Logisim tool.

2. Introduction:

A Logic Gate is assigned as an elementary building block of digital circuits. Logic gate is considered as a device which has the ability to produce one output level with the combinations of input levels. There are seven basic types of logic gates: By combining them in different ways, you will be able to implement all types of digital components. There are seven basic types of logic gates:

- AND GATE
- OR GATE
- NOT GATE
- NAND GATE
- NOR GATE
- EXCLUSIVE-OR GATE (X-OR) GATE
- EXCLUSIVE-NOR (XNOR) GATE

AND GATE

An AND gate requires two or more inputs and produce only one output. The AND gate produces an output of logic 1 state when each of the inputs are at logic 1 state and also produces an output of logic 0 state even if any of its inputs are at logic 0 state. The symbol for AND operation is '.', or we use no symbol for representing. If the inputs are of X and Y, then the output can be expressed as $Z=XY$.

2 Input AND Gate

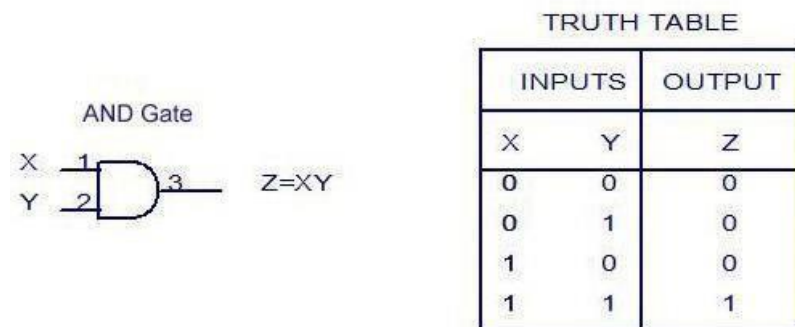


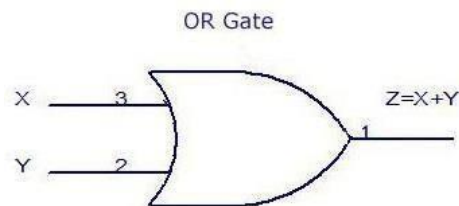
Fig 2.1: AND Gate

Table 2.1: Truth table of AND Gate

OR GATE

Similar to AND gate, an OR gate may also have two or more inputs but produce only one output. The OR gate produces an output of logic 1 state even if any of its inputs is in logic 1 state and also produces an output of logic 0 state if any of its inputs is in logic 0 state. The symbol for OR operation is '+'. If the inputs are of X and Y, then the output can be represented as $Z=X+Y$. An OR gate may also be defined as a device whose output is 1, even if one of its inputs is 1. OR gate is also called as any or all gate.

2 Input OR Gate



TRUTH TABLE

INPUTS		OUTPUT
X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

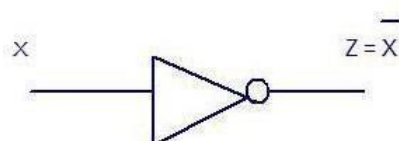
Fig 2.2: OR Gate

Table 2.2: Truth table of AND Gate

NOT GATE

The NOT gate is also called as an inverter, simply because it changes the input to its opposite. The NOT gate is having only one input and one corresponding output. It is a device whose output is always the complement of the given input. That means, the NOT gate produces an output of logic 1 state when the input is of logic 0 state and also produce the output of logic 0 state when the input is of logic 1 state.

NOT Gate



TRUTH TABLE

INPUT	OUTPUT
X	Z
0	1
1	0

Fig 2.3: NOT Gate

Table 2.3: Truth table of NOT Gate

NAND GATE:

The NAND and NOR gates are universal gates. Each of these gates can realize the logic circuits single handedly. The NAND and NOR are also called universal building blocks. Both NAND and NOR has the ability to perform three basic logic functions such as AND, OR and NOT. NAND gate is a combination of an AND gate and a NOT gate. The expression for the NAND gate is ' $\overline{XY}$ '. The output of the NAND gate is at logic 0 level only when each of the inputs assumes a logic 1 level. The truth table of two-input NAND gate is given below:

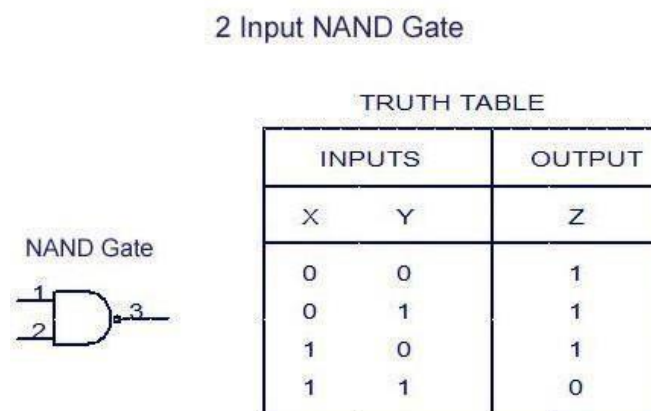


Fig 2.4: NAND Gate

Table 2.4: Truth table of NAND Gate

NOR GATE:

NOR means NOT OR. That means, NOR gate is a combination of an OR gate and a NOT gate. The output is logic 1 level, only when each of its inputs assumes a logic 0 level. For any other combination of inputs, the output is a logic 0 level. The truth table of two-input NOR gate is given below:

2 Input NOR Gate

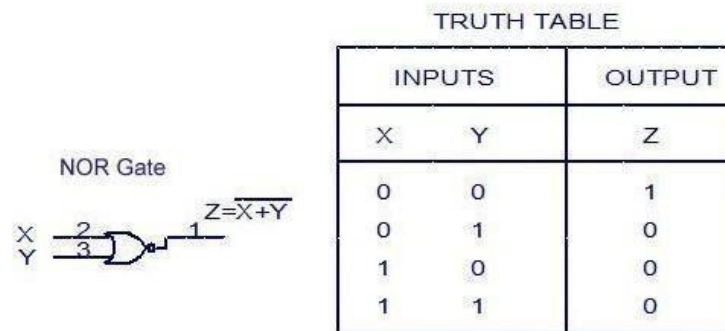


Fig 2.5: NOR Gate

Table 2.5: Truth table of NOR Gate

EXCLUSIVE-OR GATE (X-OR) GATE:

An X-OR gate is a two input, one output logic circuit. X-OR gate assumes logic 1 state when any of its two inputs assumes a logic 1 state. When both the inputs assume the logic 0 state or when both the inputs assume the logic 1 state, the output assumes a logic 0 state. The output of the X-OR gate will be the sum of the modulo sum of its inputs. X-OR gate is also termed as anti-coincidence gate or inequality detector. An X-OR gate can also be used as inverter by connecting one of the two input terminals to logic 1 and also by inputting the sequence to be inverted to the other terminal.

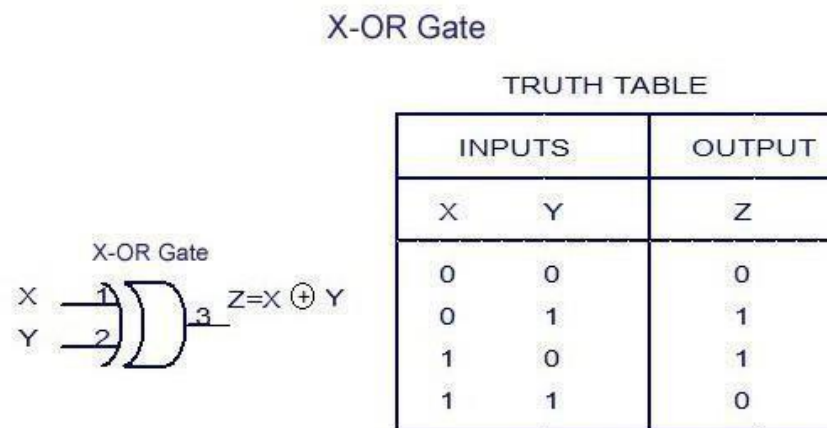


Fig 2.6: X-OR Gate

Table 2.6: Truth table of X-OR Gate

EXCLUSIVE-NOR (X-NOR) GATE

An X-NOR gate is a combination of an X-OR gate and a NOT gate. The X-NOR gate is also a two input, one output concept. The output of the X-NOR gate will be logic 1 state when both the inputs assume a 0 state or when both the inputs assume a 1 state. The output of the X-NOR gate will be logic 0 state when one of the inputs assume a 0 state and the other a 1 state. It is also named as coincidence gate, because its output will be 1 only when the inputs coincide. X-NOR gate can also be used as inverter by connecting one of the two input terminals to logic 0 and also by inputting the sequence to be inverted to the other terminal.

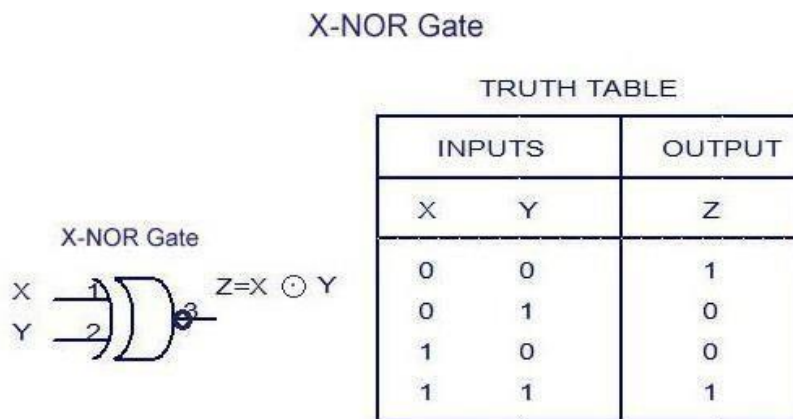


Fig 2.7: X-NOR Gate

Table 2.7: Truth table of X-NOR Gate

3. Conclusion:

Hence, we studied the Logic Gates using Logisim tool in a detailed way.

4. Reference:

1. <https://www.youtube.com/watch?v=16POVzL4wRo>
2. <https://www.geeksforgeeks.org/realization-of-logic-gate-using-universal-gates/>

Experiment No: 3

1. Aim:

Construction of half/ full adder using XOR and NAND gates and verification of its operation

2. Introduction:

Adders are digital circuits that carry out addition of numbers. Adders are a key component of arithmetic logic unit. Adders can be constructed for most of the numerical representations like Binary Coded Decimal (BCD), Excess – 3, Gray code, Binary etc. out of these, binary addition is the most frequently performed task by most common adders. Apart from addition, adders are also used in certain digital applications like table index calculation, address decoding etc.

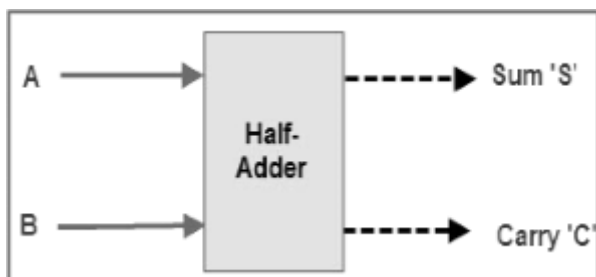
Binary addition is similar to that of decimal addition. Some basic binary additions are shown below.

0	0	1	1
<u>+0</u>	<u>+1</u>	<u>+0</u>	<u>+1</u>
0	1	1	(carry) 1 0

Fig 3.1: Schematic representation of half adder

1) Half Adder

Half adder is a combinational circuit that performs simple addition of two binary numbers. If we assume A and B as the two bits whose addition is to be performed, the block diagram and a truth table for half adder with A, B as inputs and Sum, Carry as outputs can be tabulated as follows.



Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Fig 3.2: Block diagram of Half Adder

Truth Table 3.1: Truth table of Half Adder

The sum output of the binary addition carried out above is similar to that of an Ex-OR operation while the carry output is similar to that of an AND operation. The same can be verified with help of Karnaugh Map.

The truth table and K Map simplification and logic diagram for sum output is shown below.

Truth Table		
A	B	SUM
0	0	0
0	1	1
1	0	1
1	1	0

Truth Table 3.2: Truth table of Sum

A \ B	0	1
0	0	1
1	1	0

Fig 3.3: K Map simplification for sum output of half adder

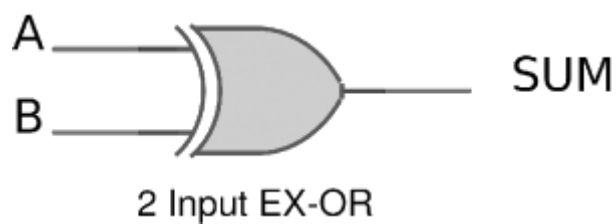


Fig 3.4: Logic diagram of Sum

$$\text{Sum} = A B' + A' B$$

The truth table and K Map simplification and logic diagram for carry is shown below.

Truth Table		
A	B	Carry
0	0	0
0	1	0
1	0	0
1	1	1

Truth Table 3.3: Truth table of Carry

A \ B	0	1
0	0	0
1	0	1

Fig 3.5: K Map simplification for carry output of half adder

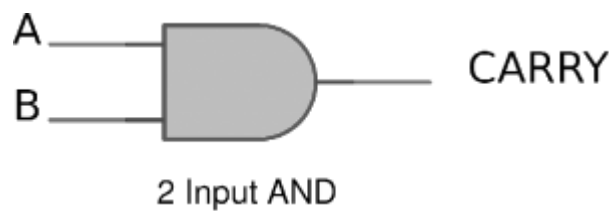


Fig 3.6: Logic diagram of Carry

$$\text{Carry} = AB$$

If A and B are binary inputs to the half adder, then the logic function to calculate sum S is Ex – OR of A and B and logic function to calculate carry C is AND of A and B. Combining these two, the logical circuit to implement the combinational circuit of half adder is shown below.

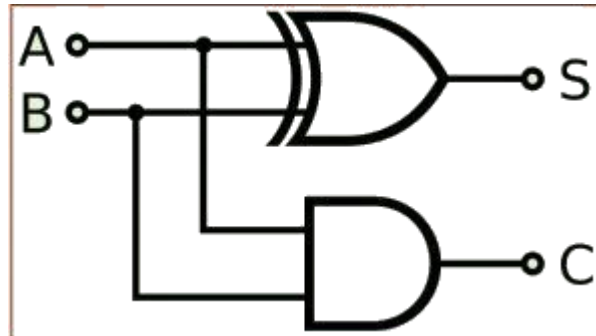


Figure 3.7: Half Adder Logic Diagram

As we know that NAND and NOR are called universal gates as any logic system can be implemented using these two, the half adder circuit can also be implemented using them. We know that a half adder circuit has one Ex – OR gate and one AND gate.

1.1) Half Adder using NAND gates

Five NAND gates are required in order to design a half adder. The circuit to realize half adder using NAND gates is shown below.

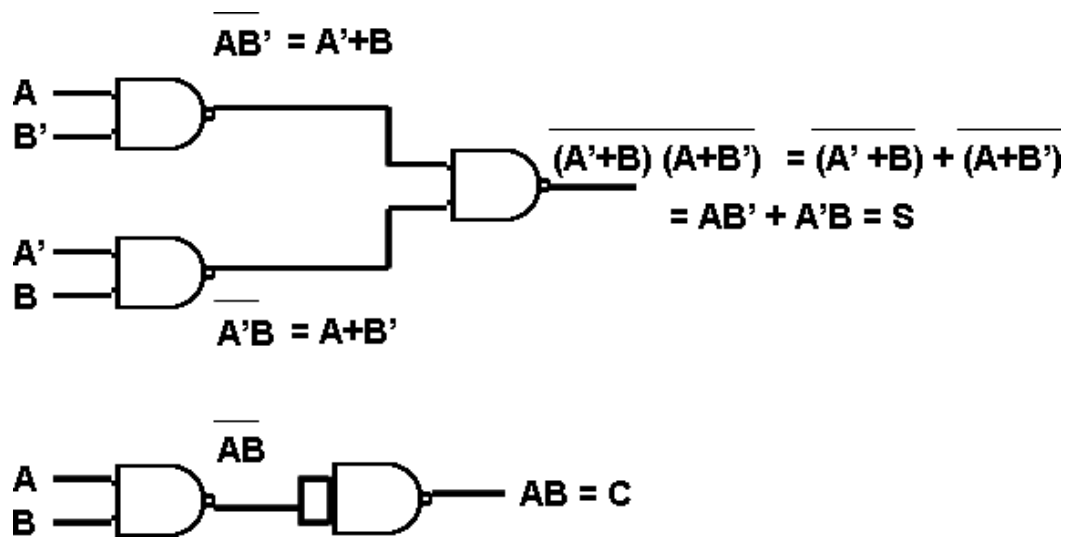


Figure 3.8: Realization of half adder using NAND gates

1.2) Half Adder using NOR gates

Five NOR gates are required in order to design a half adder. The circuit to realize half adder using NOR gates is shown below.

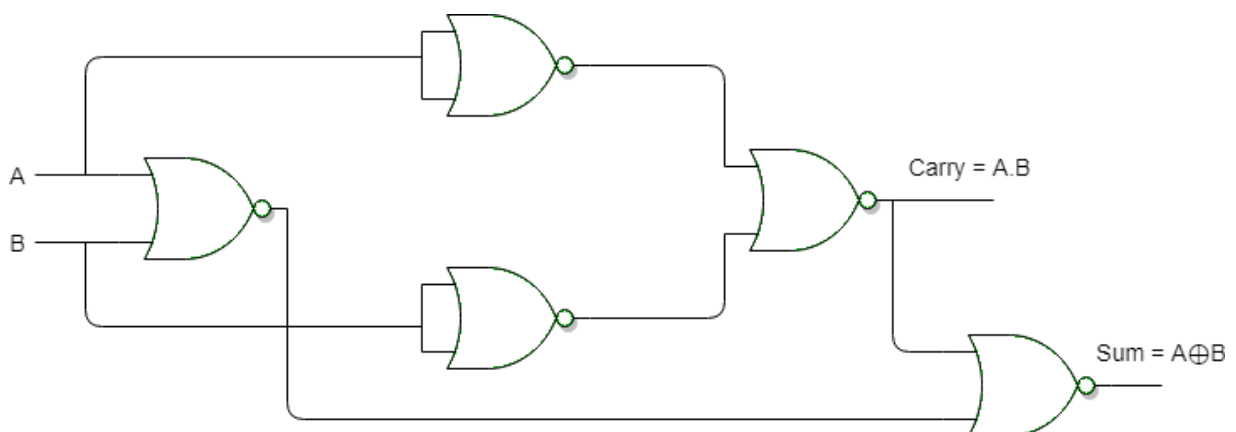


Figure 3.9: Realization of half adder using NOR Gates

2) Full Adder

Full adder is a digital circuit used to calculate the sum of three binary bits. Full adders are complex and difficult to implement when compared to half adders. Two of the three bits are same as before which are A, the augend bit and B, the addend bit. The additional third bit is carrying bit from the previous stage and is called 'Carry' – in generally represented by CIN. It calculates the sum of three bits along with the carry. The output carry is called Carry – out and is represented by Carry OUT. The block diagram of a full adder with A, B and CIN as inputs and S, Carry OUT as outputs is shown below.

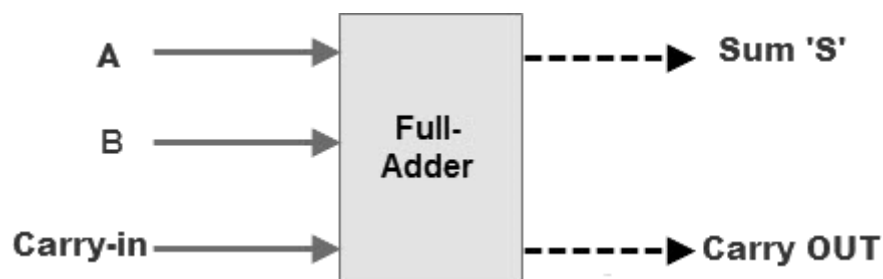


Fig 3.10: Block diagram of Full Adder

Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Truth Table 3.4: Truth table of Full Adder

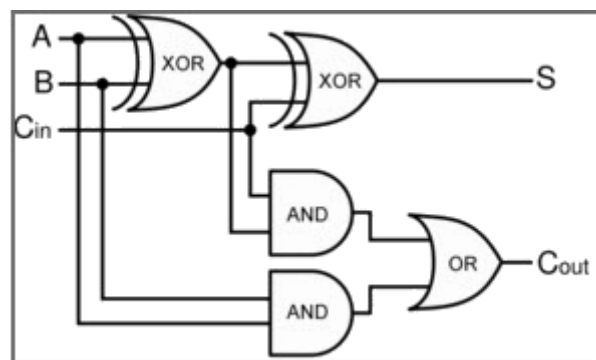
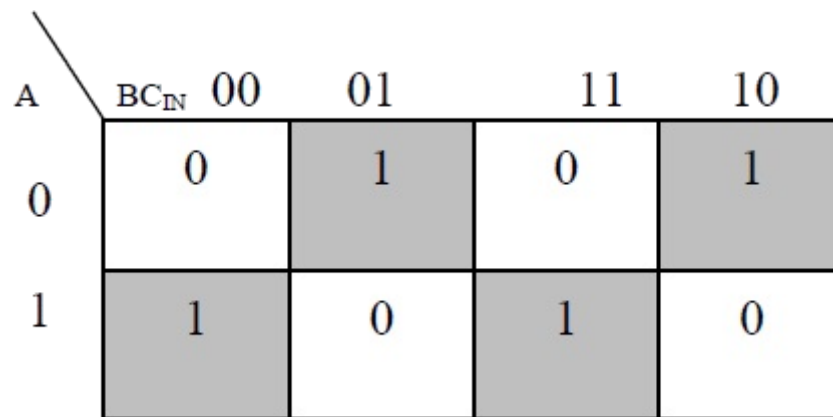


Fig 3.11: Full Adder Logic Diagram

Based on the truth table, the Boolean functions for Sum (S) and Carry – out (COUT) can be derived using K – Map.



A \ BC _{IN}	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Fig 3.12: The K-Map simplified equation for sum is $S = A'B'C_{in} + A'BC_{in}' + ABC_{in}$

A \ BC _{IN}	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Fig 3.13: The K-Map simplified equation for COUT is $COUT = AB + ACIN + BCIN$

In order to implement a combinational circuit for full adder, it is clear from the equations derived above, that we need four 3-input AND gates and one 4-input OR gates for Sum and three 2-input AND gates and one 3-input OR gate for Carry – out.

2.1) Full Adder using NAND gates

As mentioned earlier, a NAND gate is one of the universal gates and can be used to implement any logic design. The circuit of full adder using only NAND gates is shown below.

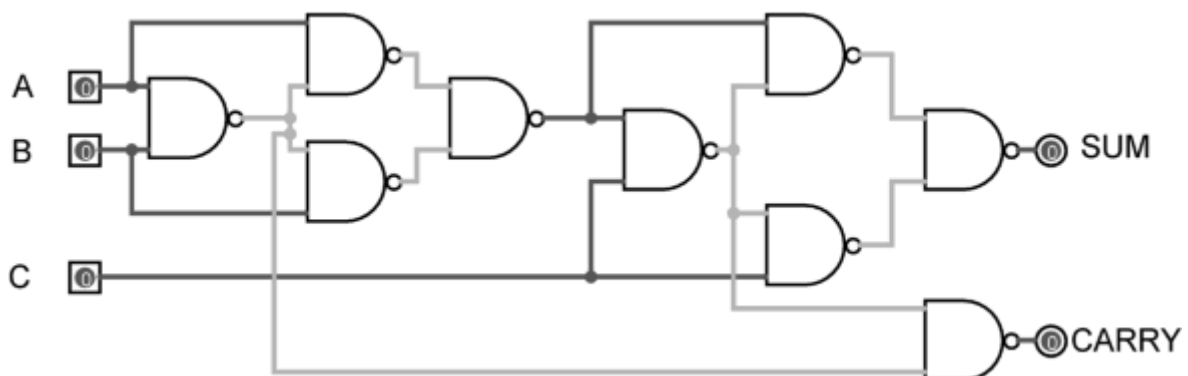


Fig 3.14: Full Adder using NAND gates

2.2) Full Adder using NOR gates

As mentioned earlier, a NOR gate is one of the universal gates and can be used to implement any logic design. The circuit of full adder using only NOR gates is shown below.

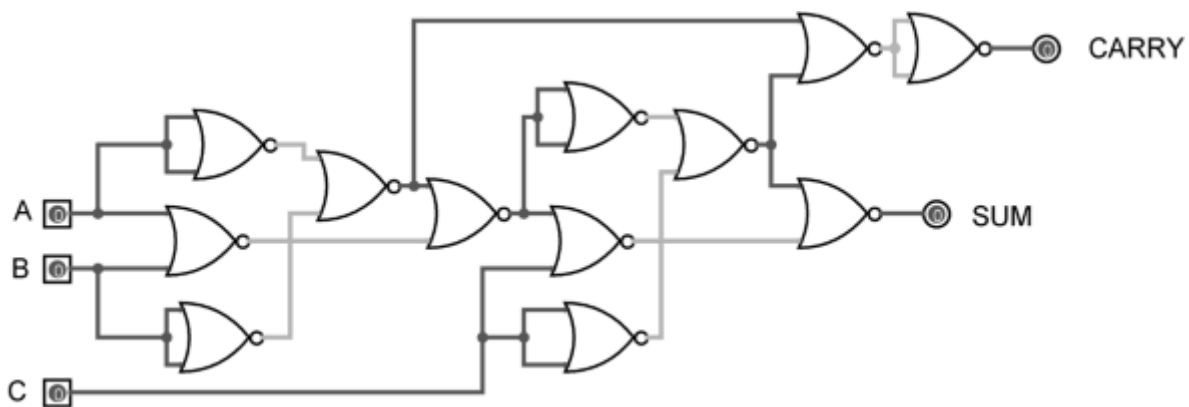


Fig 3.15: Full Adder using NOR gates

Procedure:

1) HALF ADDER

Simulator 1:



Step-1) Connect the supply(+5V) to the circuit.

Step-2) First press "ADD" button to add basic state of your output in the given table.

Step-3) Press the switches to select the required inputs "A" and "B".

Step-4) Press "ADD" button to add your inputs and outputs in the given table.

Step-5) Repeat steps 3 & 4 for next state of inputs and their corresponding outputs.

Step-6) Press the "PRINT" button after completing your simulation to get your results.

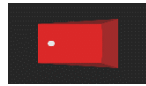
Step-7) Press the "RESET" button whenever you want to refresh your simulator.

Simulator 2:

- Step-1) Enter the Boolean input "A" and "B".
- Step-2) Enter the Boolean output for your corresponding inputs.
- Step-3) Click on "Circuit" button to check the circuit diagram for half adder.
- Step-4) Click on "Check" Button to verify your output.
- Step-5) Click "Print" if you want to get print out of Truth Table.
- Step-6) Click on "Reset" button if you want to reset input and outputs.

2) FULL ADDER

Simulator 1:



- Step-1) Connect the supply(+5V) to the circuit.
- Step-2) First press "ADD" button to add basic state of your output in the given table.
- Step-3) Press the switches to select the required inputs "A" and "B" and "C_{in}".
- Step-4) Press "ADD" button to add your inputs and outputs in the given table.
- Step-5) Repeat steps 3 & 4 for next state of inputs and their corresponding outputs.
- Step-6) Press the "PRINT" button after completing your simulation to get your results.
- Step-7) Press the "RESET" button whenever you want to refresh your simulator.

Simulator 2:

- Step-1) Enter the Boolean input "A" and "B" and "C_{in}".
- Step-2) Enter the Boolean output for your corresponding inputs.
- Step-3) Click on "Circuit" button to check the circuit diagram for full adder.
- Step-4) Click on "Check" Button to verify your output.
- Step-5) Click "Print" if you want to get print out of Truth Table.
- Step-6) Click on "Reset" button if you want to reset input and outputs.

INSTRUCTIONS

- 1) Connect the Supply(+5V) to the IC.
- 2) Press the switches for inputs "A" and "B".

The switch in ON state is  and the switch in OFF state is .

- 3) The SUM LED glows if any one switch is ON out of both the switches else it won't glow.

And the CARRY LED glows if the both switch's state is  else it will remain OFF in other 3 states.

- 4) Press "ADD" button to add your inputs and outputs in the given table.
- 5) Repeat steps 3 & 4 for next state of inputs and their corresponding outputs.
- 6) Press the "PRINT" button after completing your simulation to get your results.
- 7) Press the "RESET" button whenever you want to refresh your simulator.

Half adder Output:

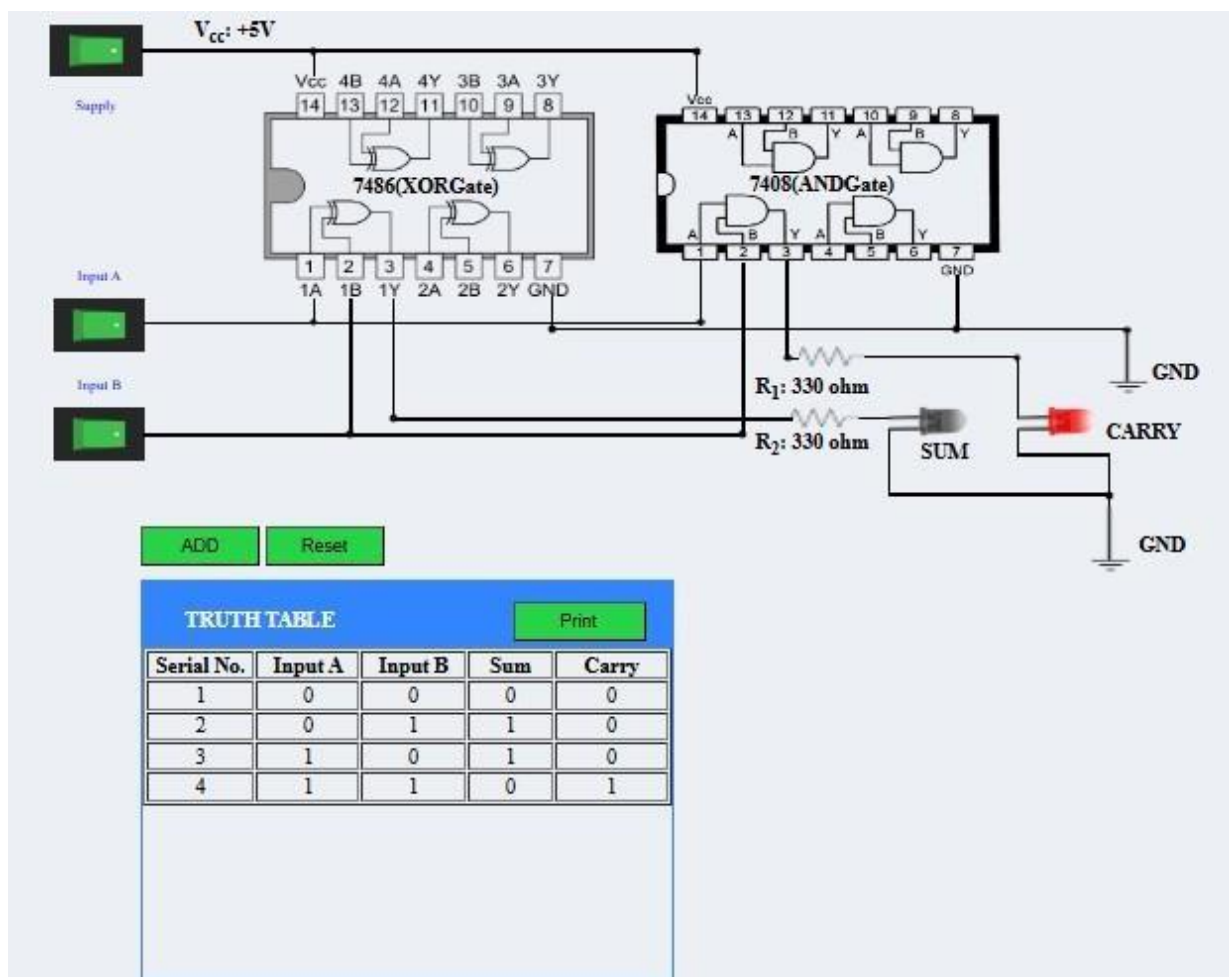


Fig 3.16: Half Adder circuit with the help of LEDs

Full adder Output:

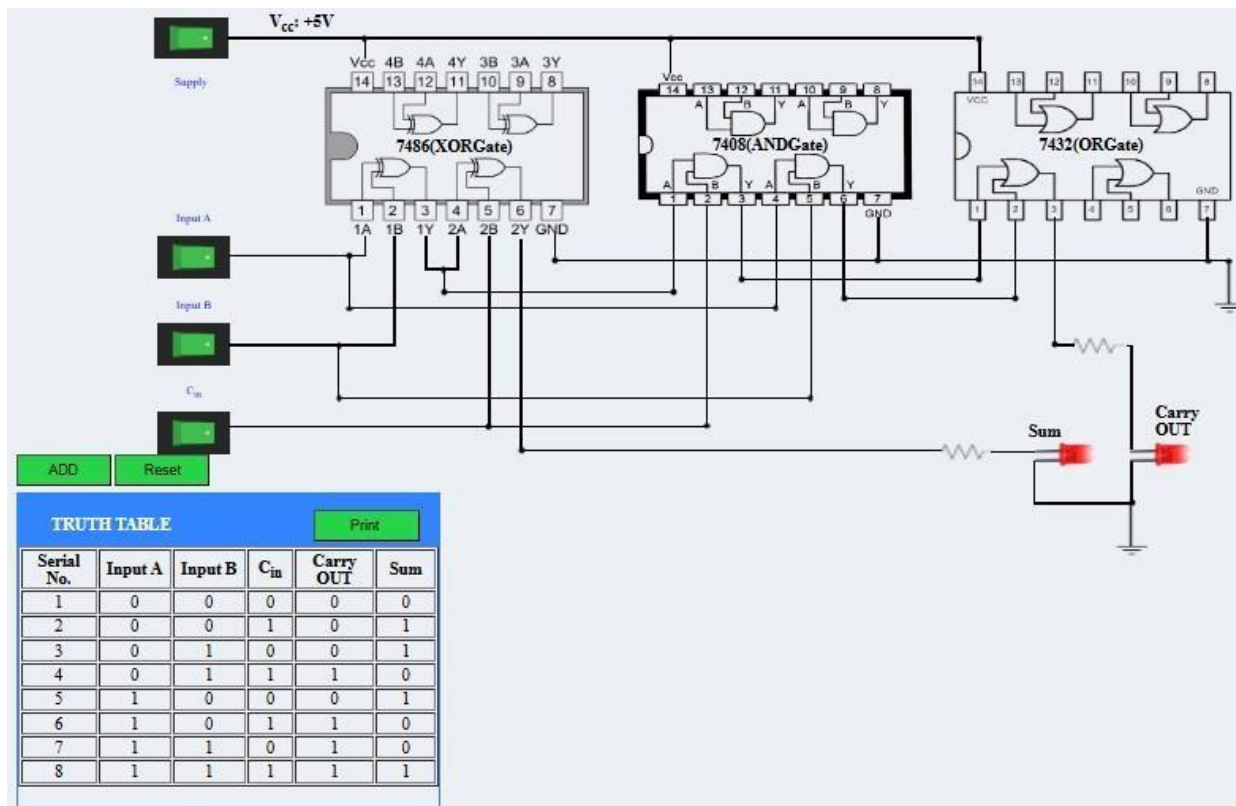


Fig 3.17: Full Adder circuit with the help of LEDs

3. Conclusion:

Hence, we studied the Construction of half/ full adder using XOR and NAND gates and verification of its operation in a detailed way.

4. Reference:

1. <https://de-iitr.vlabs.ac.in/exp/half-full-adder/theory.html>

Experiment No: 4

1. Aim:

To verify the truth table of half subtractor by using the ICs of XOR, NOT and AND gates and of full subtractor by using the ICs of XOR, AND, NOT and OR gates respectively and analyze the working of half subtractor and full subtractor circuit with the help of LEDs in simulator 1 and verify the truth table only of half subtractor and full subtractor in simulator 2.

2. Introduction:

- Subtractor circuits take two binary numbers as input and subtract one binary number input from the other binary number input. Similar to adders, it gives out two outputs, difference and borrow (carry-in the case of Adder). There are two types of subtractors.
- Half Subtractor
- Full Subtractor

Half Subtractor

The half-subtractor is a combinational circuit which is used to perform subtraction of two bits. It has two inputs, A (minuend) and B (subtrahend) and two outputs Difference and Borrow. The logic symbol and truth table are shown below.

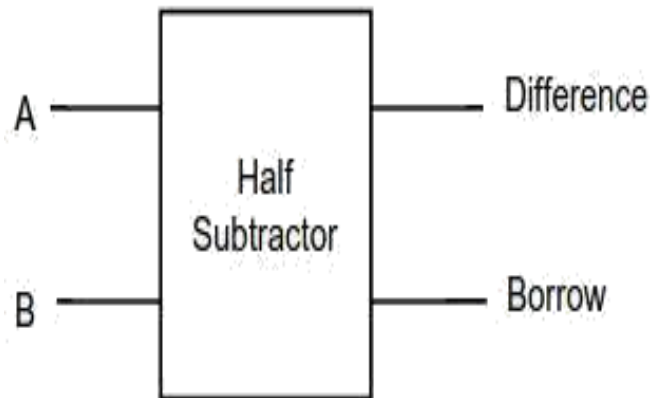


Fig 4.1: Logic Symbol of Half subtractor

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Truth table 4.1: Truth Table of Half subtractor

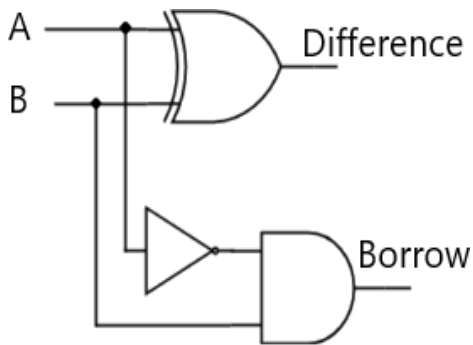


Fig 4.2: Circuit Diagram of Half subtractor

From the above truth table, we can find the boolean expression. Difference = $A \oplus B$

Borrow = $A' B$

From the equation we can draw the half-subtractor circuit as shown in the figure 4.2.

2) Full Subtractor

A full subtractor is a combinational circuit that performs subtraction involving three bits, namely A (minuend), B (subtrahend), and Bin (borrow-in). It accepts three inputs: A (minuend), B (subtrahend) and a Bin (borrow bit) and it produces two outputs: D (difference) and Bout (borrow out). The logic symbol and truth table are shown below.

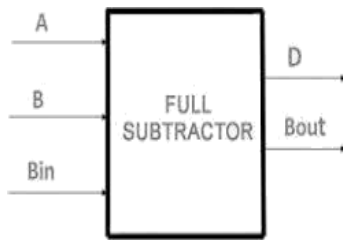


Fig 4.3: Logic Symbol of Full subtractor

A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Truth Table 4.2: Truth Table of Full subtractor

From the above truth table, we can find the boolean

expression. $D = A \oplus B \oplus B_{in}$

$B_{out} = A' B_{in} + A' B + B B_{in}$

From the equation we can draw the Full-subtractor circuit as shown in the figure 4.4.

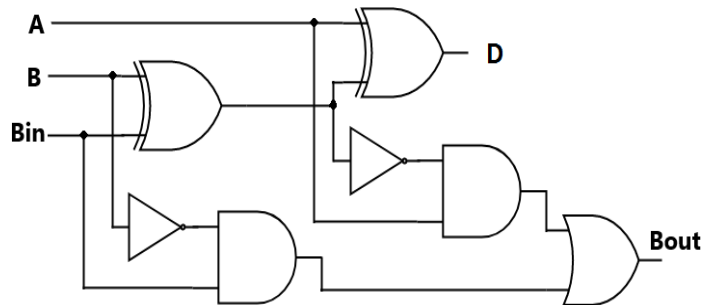


Fig 4.4: Circuit Diagram of Full subtractor

Procedure:

1) HALF SUBTRACTOR

Simulator 1:



Step-1) Connect the Supply(+5V) to the circuit.

Step-2) First press "ADD" button to add basic state of your output in the given

table. Step-3) Press the switches to select the required inputs "A" and "B".

Step-4) Press "ADD" button to add your inputs and outputs in the given table.

Step-5) Repeat steps 3&4 for next state of inputs and their corresponding outputs.

Step-6) Press the "PRINT" button after completing your simulation to get your results.

Simulator 2:

Step-1) Enter the Boolean input "A" and "B".

Step-2) Enter the Boolean output for your corresponding

inputs. Step-3) Click on "Check" Button to verify your output.

Step-4) Click "Print" if you want to get print out of Truth

Table. Step-5) Click "Reset" if you want to reset inputs and outputs.

1) FULL SUBTRACTOR

Simulator 1:



Step-1) Connect the Supply(+5V) to the circuit.

Step-2) First press "ADD" button to add basic state of your output in the given

table. Step-3) Press the switches to select the required inputs "A" and "B" and "Bin".

Step-4) Press "ADD" button to add your inputs and outputs in the given table.

Step-5) Repeat steps 3&4 for next state of inputs and their corresponding outputs.

Step-6) Press the "PRINT" button after completing your simulation to get your results.

Simulator 2:

Step-1) Enter the Boolean inputs "A" and "B" and "Bin".

Step-2) Enter the Boolean output for your corresponding

inputs. Step-3) Click on "Check" Button to verify your output.

Step-4) Click "Print" if you want to get print out of Truth

Table. Step-5) Click "Reset" if you want to reset inputs and outputs.

Half Subtractor

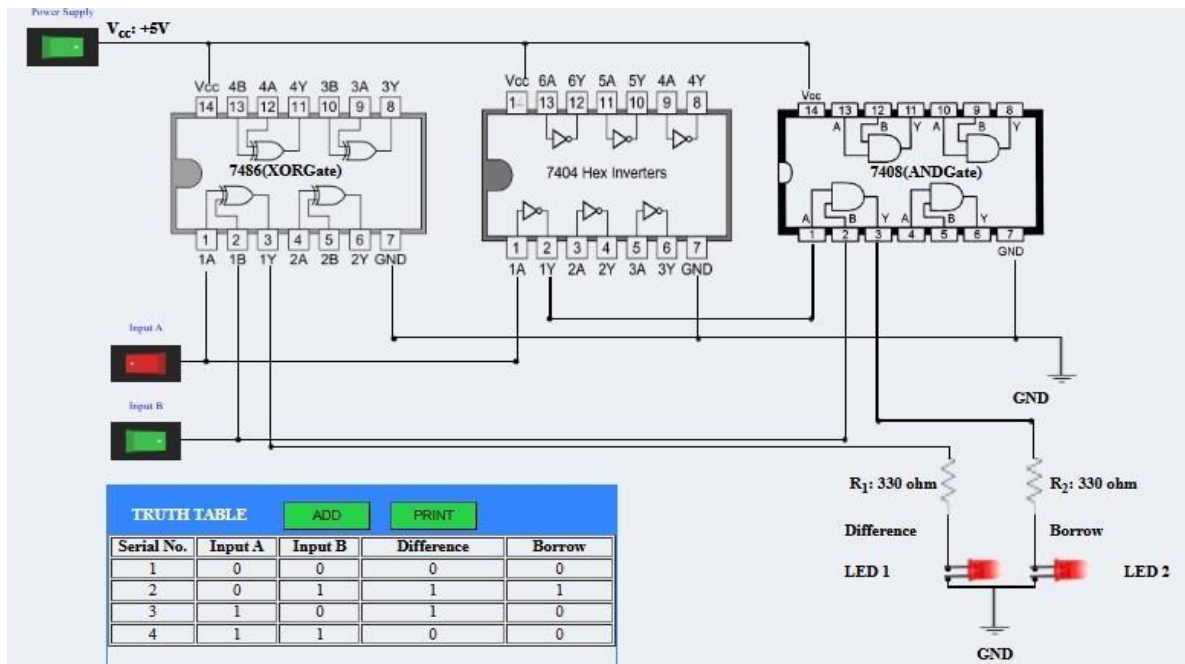


Fig 4.5: Half subtractor circuit with the help of LEDs

Full Subtractor

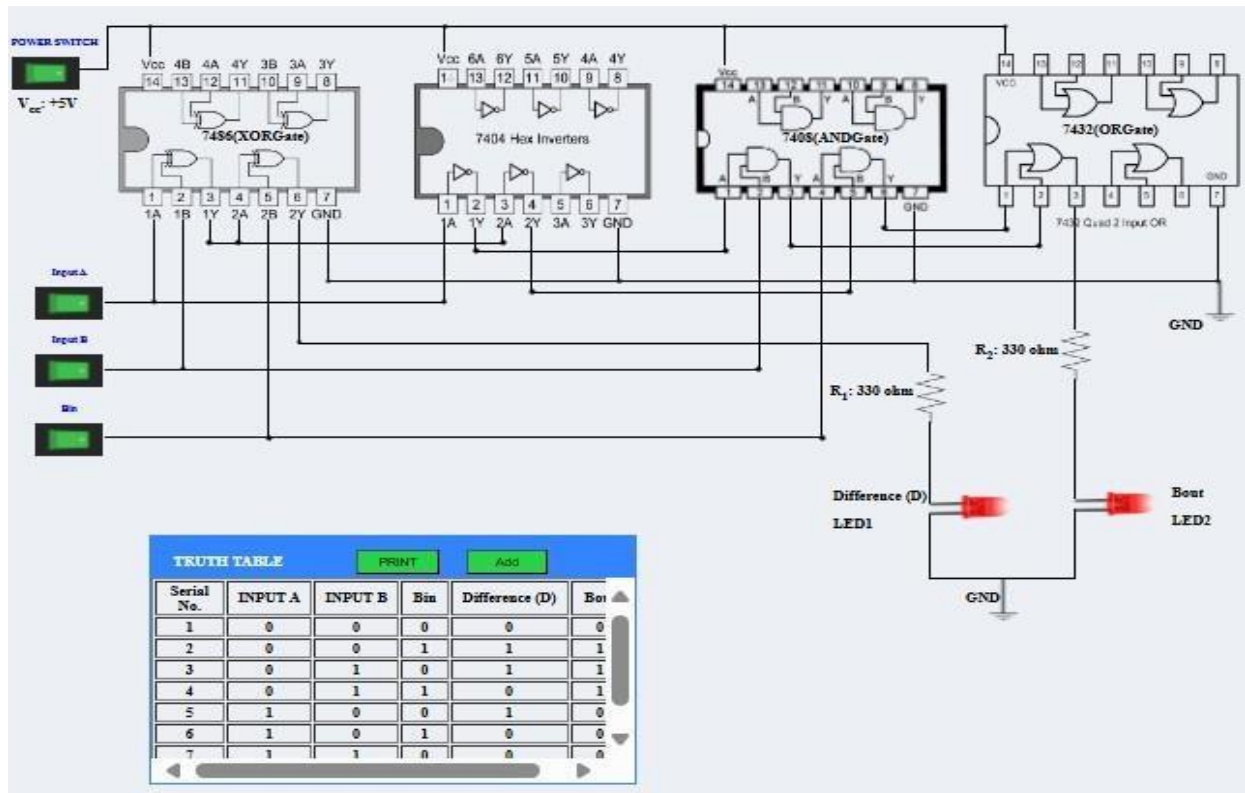


Fig 4.6: Full subtractor circuit with the help of LEDs

3. Conclusion:

Hence, we verified the truth table of half subtractor by using the ICs of XOR, NOT and AND gates and of full subtractor by using the ICs of XOR, AND, NOT and OR gates in a detailed way.

4. Reference:

- <https://de-iitr.vlabs.ac.in/exp/half-full-subtractor/theory.html>